

# CS482/495/496 Software Project: Artificial Axon Pipeline

Chloe Miranda and Jonathan Dargakis

2026-05-06

## 1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

1. Client name: Dr.Alexis Lowe
2. Client title: Senior Scientist
3. Client email address: alexislowe816@gmail.com
4. Client employer: Artificial Axon Labs
5. How you know the client: I was mutually connected with them through a colleague based on our shared interest in computational medicine.

## 2 Project Description

### 2.1 Overview

Artificial Axon Labs specializes in studying artificial neurons and their myelination using high-resolution 3D imaging. Currently, Dr. Lowe and Dr. Jagelskia rely on a highly fragmented and inefficient workflow to process these image datasets. In its present form, their pipeline spans Python scripts, Jupyter notebooks, and ImageJ each responsible for different stages of preprocessing, rendering, and analysis. Switching between these tools for every dataset is tedious and time-consuming, and even minor adjustments often require manually editing code or scripts. Because each dataset may require unique parameter settings, researchers must repeatedly tune and reconfigure files in different environments, significantly slowing down their ability to produce usable figures and scientific insights.

Additionally, the current workflow suffers from decentralized practices between team members. Dr. Lowe and Dr. Jagelskia process their data using different methods, scripts, and organizational structures, resulting in inconsistent outputs and further complicating collaboration. The lack of a unified system not only increases cognitive load but also makes reproducibility and long-term data management more difficult.

The purpose of this project is to design and implement a dedicated software application that consolidates this entire image-processing pipeline into a single, cohesive system, as well as utilizes computational techniques to speed up the process of rendering their 3D data. The goal is not to eliminate scientific control, but to provide a streamlined interface where researchers can upload files, adjust analysis parameters, view results, and export processed images without navigating three separate programs. By integrating configurable image-analysis tools and automated processing routines, the application aims to reduce repetitive work while preserving the flexibility required for scientific experimentation.

A key requirement expressed by the client is the ability to maintain a history of processed datasets. Both researchers emphasized the importance of comparing multiple renderings, tracking different configurations, and easily retrieving past results for publication-quality figures. The application will therefore include a

structured history or “edit-tracking” system that allows users to revisit previous outputs or apply past settings to new datasets.

Ultimately, this project addresses a real and immediate bottleneck in the client’s research process. By centralizing tools, reducing manual overhead, and supporting reproducible workflows, the software will significantly improve efficiency and consistency in analyzing artificial neuron imagery—benefits that are vital for a rapidly growing research lab.

## **2.2 Key Features**

### **2.3 File Management: image files are organized by wells/sections of the 3d printed axon.**

- upload files to app
- automated file organization based on naming conventions

### **2.4 Run configuration and history:**

Runs of file processing will have options, and those choices can be recorded and reused for future runs.

- Each run will also have some sort of metadata attached to it, including the configuration used, date, and ID
- Configurations will consist of:
  - input and output folder paths
  - image data type: 2D or 3D
  - microscope type: Keyence or Olympus
  - Channel settings
  - number of field of views per well
- Note that for 3D data, additional inputs are required
  - number of frames per z-stack
  - cherry picking frames for the run
  - distance between frames (micrometers)
  - Yes or no: run wrapping analysis algorithm

### **Automated Image Processing:**

Due to the nature of their work, our clients’ pipeline will remain relatively intact, utilizing a combination of bash, python and ImageJ to process these images but now with user inputs and consolidated into one run.

### **Result management:**

- after running the scripts, there should be three general outputs:
  - tiff files of various masks for each image
  - data files of each field of view
  - summary data of each well used in experiment
  - tiff file of mylein overlap
- These four outputs should be consolidated into two separate csv files, one for well data and one for field of view data.

- For 3D data, there should also be an extra csv for the outputs of wrapping analysis.
- The metadata and run configuration should also be recorded and output somewhere, likely as either a text file or in the app itself

## 2.5 Why this Project is Interesting

This project is interesting because it serves as a bridge between biology and computer science, applying computational tools to address real problems in modern biomedical research. The software directly supports the study of artificial neurons and myelination—processes that are essential for understanding neural development, neurodegenerative diseases, and the behavior of complex neural systems. By building tools that manage large image datasets and extract meaningful information from 3D microscopy images, the project contributes to a field where efficient data handling is often a major bottleneck.

Beyond the scientific relevance, this project is enticing because it involves solving multifaceted engineering challenges. It blends file system design, image processing, data visualization, and user-facing tool development. A challenge I'm excited to tackle with this project is the UI design. Our clients need to have quite a lot of control over what's being processed so it will be interesting to figure out a UI that allows that control without revealing too much of the backend and overwhelming them with information. Developing robust software that researchers can rely on requires disciplined software engineering practices, thoughtful UX, and careful consideration of performance and scalability—skills that align strongly with the goals of a capstone.

Finally, this project matters because the underlying research matters. Improving tools for analyzing 3D neuron imagery has immediate value for the professionals using them and the scientific questions they are trying to answer. The work that Artificial Axon Labs is doing is already groundbreaking. Good data processing pipelines can accelerate the pace of discovery, reduce human error, and make previously impractical analyses feasible. A capstone that enables real researchers to work more efficiently has intellectual and practical significance.

## 2.6 Summary of Efforts to Find a Project

I was mutually connected with Dr. Lowe through a community crochet night. Other connections I've reached out to did not work out or were not able to offer anything.

# 3 Requirements

## 3.1 Non-Functional Requirements

| ID   | Title                | Category        | Description                                                                                                                   |
|------|----------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------|
| NFR1 | Color scheme         | Visual / Design | The application shall use a dark red and white color scheme.                                                                  |
| NFR2 | Error messages       | Visual / Design | The system shall display clear and informative error messages whenever a script fails during pipeline execution.              |
| NFR3 | Editable codebase    | Accessibility   | The codebase shall be easily editable to support future requirements and extensions.                                          |
| NFR4 | Analysis consistency | Results         | The system shall always produce standard wrapping analysis results using the original formula for all 3D data.                |
| NFR5 | Exporting on error   | Visual / Design | Options to export scripts as text files and save settings shall be presented whenever an error of both user and system occur. |

| <b>ID</b> | <b>Title</b>           | <b>Category</b> | <b>Description</b>                                                                                                                 |
|-----------|------------------------|-----------------|------------------------------------------------------------------------------------------------------------------------------------|
| NFR6      | User confirmations     | Visual / Design | The system shall display confirmation dialogs when starting, ending, or deleting a process.                                        |
| NFR7      | Pipeline performance   | Performance     | The system shall execute pipeline sessions within a reasonable time under normal hardware conditions for both 2D and 3D pipelines. |
| NFR8      | File management safety | Reliability     | The system shall not modify or delete original input files; all changes shall occur within system-managed copies.                  |
| NFR9      | Reproducibility        | Reproducibility | The system shall produce identical outputs when executed multiple times using the same inputs and configuration settings.          |
| NFR10     | User instructions      | Visual / Design | The system shall provide tooltips or guidance for all major configuration options and user operations.                             |

note: In all user stories and system requirements, unless the pipeline is explicitly stated, assume that both 2D and 3D pipelines are included.

### 3.2 User Stories

| <b>ID</b> | <b>Title</b>                        | <b>Points</b> | <b>Description</b>                                                                                                                    |
|-----------|-------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------|
| U1        | Data upload                         | 1             | As a user, I want to upload data files or folders so that the application can access data for processing.                             |
| U2        | Setting inputs                      | 2             | As a user, I want to input settings for data processing so that I can customize how data is processed depending on the experiment.    |
| U3        | Channel selection                   | 1             | As a user, I want to select one or multiple channels in a 2D experiment so that I can perform simple or complex experiments.          |
| U4        | Image preview after organization    | 2             | As a user, I want to preview selected images after organization so that I can determine which images should be processed.             |
| U5        | Selecting images after organization | 1             | As a user, I want to select organized images for a run so that I can clean up experimental image datasets without reorganizing files. |
| U6        | Deleting images after organization  | 1             | As a user, I want to delete organized images for a run so that I can clean up experimental image datasets without reorganizing files. |
| U7        | 2D or 3D pipelines                  | 1             | As a user, I want to choose between 2D and 3D image pipelines so that I can correctly process different types of image data.          |
| U8        | Intensity threshold                 | 1             | As a user, I want to set an intensity threshold or enable automatic selection so that I can subtract debris or noise from images.     |
| U9        | Field of view amount                | 1             | As a user, I want to specify the number of fields of view per well so that I can define how many wells are used in an experiment.     |

| <b>ID</b> | <b>Title</b>                   | <b>Points</b> | <b>Description</b>                                                                                                                   |
|-----------|--------------------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------|
| U10       | Channel selection              | 1             | As a user, I want to select which channels are used so that I can identify stains and suppress debris.                               |
| U11       | Microscope selection           | 1             | As a user, I want to select which microscope was used so that the system can distinguish between Keyence and Olympus data.           |
| U12       | Creating setting configuration | 2             | As a user, I want to create configuration profiles for data processing sessions so that I can reuse them for similar experiments.    |
| U13       | Reading setting configuration  | 2             | As a user, I want to view saved configuration profiles so that I can reuse previous settings.                                        |
| U14       | Update setting configuration   | 2             | As a user, I want to update saved configurations so that I can modify them to meet new experimental needs.                           |
| U15       | Delete setting configuration   | 2             | As a user, I want to delete saved configurations so that I can manage and declutter my configuration list.                           |
| U16       | Z-stack frames                 | 1             | As a user, I want to enter the number of frames per z-stack so that I can define the structure of 3D image data.                     |
| U17       | Frame distance                 | 1             | As a user, I want to enter the distance between frames in micrometers so that I can define spacing in 3D image data.                 |
| U18       | Text file generation           | 2             | As a user, I want to generate text files of scripts so that I can manually edit and run them if the system fails.                    |
| U19       | Preview configuration          | 1             | As a user, I want to preview metadata of saved configurations so that I can identify the correct configuration before selecting it.  |
| U20       | Preview cherry-pick images     | 1             | As a user, I want to preview images before selecting them for 3D processing so that I can avoid skewed data.                         |
| U21       | Session start                  | 1             | As a user, I want to start a data processing session so that selected images and settings execute as a single run.                   |
| U22       | Session end                    | 2             | As a user, I want to stop or end a running session so that I can terminate or finalize processing when needed.                       |
| U23       | Session history                | 2             | As a user, I want to view metadata from past sessions so that I can review and document previous runs.                               |
| U24       | Choosing configurations        | 1             | As a user, I want to select a saved configuration before starting a run so that I can save time.                                     |
| U25       | Auto-fill                      | 1             | As a user, I want to automatically run the pipeline after selecting a configuration so that I can avoid stepping through each stage. |
| U26       | Rerun                          | 1             | As a user, I want to auto-fill settings from a previous configuration so that I can rerun experiments with minimal changes.          |

| ID                  | Title                | Points    | Description                                                                                                             |
|---------------------|----------------------|-----------|-------------------------------------------------------------------------------------------------------------------------|
| U27                 | Pause a run          | 1         | As a user, I want to pause or stop a run so that I can halt execution if incorrect settings were chosen.                |
| U28                 | Image recovery       | 1         | As a user, I want to undo image exclusions so that I can recover images mistakenly removed from the 3D pipeline.        |
| U29                 | Progress monitor     | 1         | As a user, I want to see which stage the pipeline is currently executing so that I can track processing progress.       |
| U30                 | Export failed files  | 3         | As a user, I want to export scripts with previous settings so that I can manually run the pipeline if the system fails. |
| U31                 | Preview results      | 2         | As a user, I want to preview results before ending a session so that I can verify outputs.                              |
| U32                 | Preview graphs       | 5         | As a user, I want to generate graphs before exporting results so that I can visualize and verify data.                  |
| U33                 | SSH access for files | 1         | As a user, I want to access my files via SSH so that I can retrieve them directly.                                      |
| U34                 | Mask creation        | 2         | As a user, I want to create masks for both 2D and 3D images so that I can improve visibility and analysis quality.      |
| <b>Total Points</b> |                      | <b>51</b> |                                                                                                                         |

### 3.3 System Requirements

| ID | Title                     | Points | Description                                                                                                            |
|----|---------------------------|--------|------------------------------------------------------------------------------------------------------------------------|
| S0 | File organization         | 2      | The pipeline will create a uniform file structure when accepting raw TIFF and OIR files.                               |
| S1 | File renaming             | 1      | The pipeline will automatically rename image data files to create uniform scaffolding for future processing steps.     |
| S2 | Field of view aggregation | 1      | The 2D pipeline will aggregate data from all fields of view into a single summary file when the pipeline is complete.  |
| S3 | Wrapping analysis         | 1      | The 3D pipeline will generate a wrapping analysis when the pipeline is complete.                                       |
| S4 | Ezra's algorithm          | 2      | The 3D pipeline will run Ezra's algorithm when selected in the settings.                                               |
| S5 | Start date and time       | 1      | The pipeline will automatically record the starting date and time of a data processing session when a session begins.  |
| S6 | Script runtime            | 1      | The pipeline will automatically record the runtime of scripts executed during a session, only when a script is run.    |
| S7 | End date and time         | 1      | The pipeline will record the date and time when a session finishes after producing output and being ended by the user. |
| S8 | Debris calculation        | 1      | The 2D pipeline will calculate debris present in images when the 2D pipeline is selected.                              |

| ID                  | Title                     | Points    | Description                                                                                                                         |
|---------------------|---------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| S9                  | Overlapping calculation   | 2         | The pipeline will calculate overlapping myelin present in images when either the 2D or 3D pipeline is selected.                     |
| S10                 | Myelin growth calculation | 2         | The pipeline will calculate myelin growth on artificial axons found in images after overlapping myelin is calculated.               |
| S11                 | Calculation results       | 4         | The pipeline will consolidate calculation results into appropriate CSV files after all calculations are performed.                  |
| S12                 | Ezra's results            | 1         | Ezra's algorithm will consolidate its results with the main CSV file after being selected and all other calculations are completed. |
| S13                 | Session history           | 1         | The pipeline will record which pipeline steps were executed and which steps failed during a session run.                            |
| S14                 | Input validation          | 1         | The system will validate all user-provided inputs to prevent invalid or out-of-range values.                                        |
| S15                 | Graceful failure handling | 2         | The pipeline will allow partial results to be preserved if a processing stage fails.                                                |
| <b>Total Points</b> |                           | <b>25</b> |                                                                                                                                     |

## 4 System Design

### 4.1 Architecture

This project adopts a hybrid architecture that combines Pipeline and Model–View–Controller (MVC) patterns to balance modular data processing with a clean, maintainable user interface structure.

The pipeline architecture is used to structure the core processing workflow, where data flows through a series of well-defined stages. Each stage performs a specific transformation or analysis and passes its output to the next stage. This approach improves modularity, simplifies debugging, and allows individual components to be modified or extended without affecting the rest of the system. It also allows our client to have a high-level understanding of their own data processing session when it runs.

The MVC architecture is applied to the user interface and front-end components, particularly for managing and reusing saved configurations. The Model represents application data and configuration states, the View handles user-facing presentation, and the Controller manages user interactions and coordinates updates between the Model and View. The controller will also manage file retrieval for any selection and preview thumbnails. This separation of concerns improves maintainability, supports clearer organization of UI logic, and enables future expansion of features with minimal coupling.

Together, these architectural patterns provide a scalable foundation that supports both efficient data processing and a flexible, user-friendly interface.

## 4.2 Diagrams

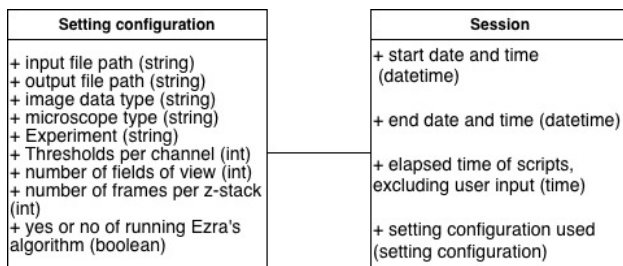


Figure 1: Class diagram of proposed objects, setting configurations and settings

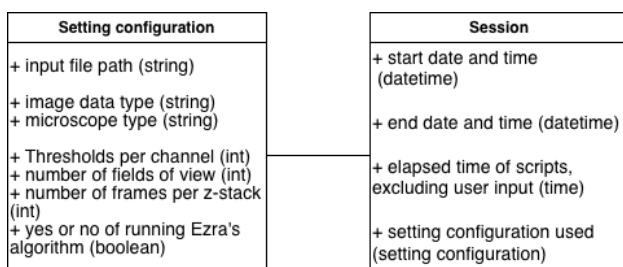


Figure 2: Updated Class Diagram as of Iteration 4.

Updated diagram note: no more output file paths or experiment strings. no need for either as both are automatically determined or simply unnecessary.

## 4.3 Technology

This project will be reflecting their original pipeline's code The primary programming language is Python, which serves as the core language for application logic, orchestration, and integration between components. Bash scripts are used to automate pipeline execution and manage system-level tasks. We'll also be using Python's datetime library to store date and time objects for session history.

For the user interface, the project uses Tkinter, a lightweight Python library for building graphical user interfaces. Previously, their pipeline used a separate application, ImageJ to do any image processing. Instead, we will use a python library known as pyimagej which allows ImageJ functionality to be accessed and controlled directly from Python code. This enables seamless coordination between image processing tasks and the rest of the application logic.

All testing is conducted using pytest which provides a structured and automated testing framework to validate individual components and ensure overall system correctness. Data generated and consumed by the application is stored locally as per the client's request as well as general sanity.

## 4.4 Coding Standards

general coding standards

- Code should prioritize readability and clarity over clever or overly compact implementations.
- Follow consistent naming conventions across all files, functions, and variables.
- Avoid duplication by reusing functions and scripts where possible.
- All code must be version-controlled and include meaningful commit messages.



- All commits must be linked to a respective task in github via its commit message
- use UPPER\_SNAKE\_CASE for constants
- use lower\_snake\_case for local variables
- keep python and bash scripts in separated directories
- have meaningful file names

python coding standards

- use snake\_case for variables and functions
- use PascalCase for class names
- include docstrings for all modules, classes, and public functions.
- include explicit error handling with try/except statements

bash scripting standards

- make sure all scripts begin with a shebang.
- avoid hard-coded paths. Opt for variables instead

testing and validation standards

- unless you physically can't, all code should be tested
- tests should be repeatable and not depend on an external state unless explicitly simulated to do so

## 4.5 Data

The main data structures that will be stored are setting configurations and sessions:  
setting configuration are comprised of:

- input file path (string)
- image data type (string)
- microscope type (string)
- Experiment (string)
- Thresholds per channel (int)
- number of fields of view (int)
- number of frames per z-stack (int)
- yes or no of running Ezra's algorithm (boolean)

a session will contain the following:

- start date and time (datetime)
- end date and time (datetime)
- elapsed time of scripts (excluding user input)
- setting configuration used (configuration object)

all data objects will be stored as simple JSON files in local file storage, not utilizing a database.

## 4.6 UI Mocks

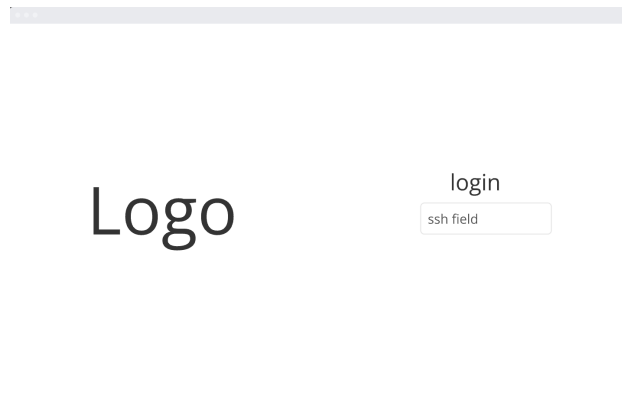


Figure 3: login page for ssh

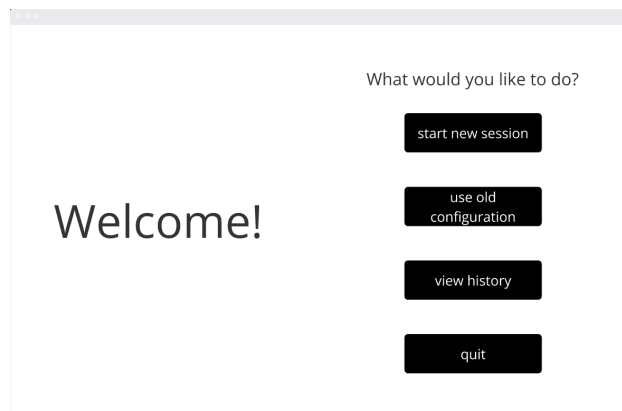


Figure 4: landing page after login

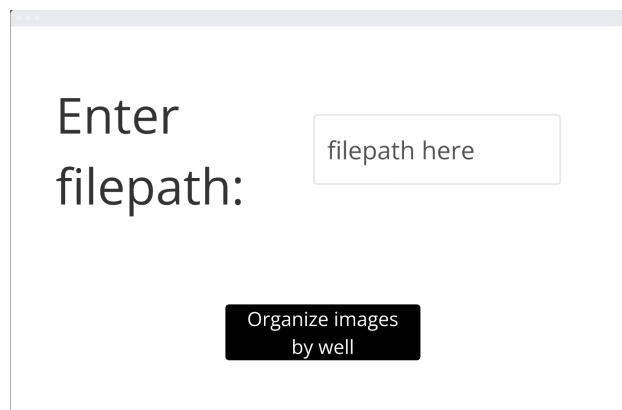


Figure 5: image data filepath input menu to access image data files for session

image type: ☐ 2D ☐ 3D

microscope: ☐ Keyence ☐ Olympus

experiment:

thresholds:  ☐ Auto

fields of view:

Figure 6: 2D setting inputs

image type: ☐ 2D ☐ 3D

microscope: ☐ Keyence ☐ Olympus

experiment:

thresholds:  ☐ Auto

fields of view:

|            |
|------------|
| DAPI       |
| GFP-myelin |
| CY5-myelin |
| GFP-debris |
| CY5-debris |

Figure 7: dropdown menu example using 2D image

image type: ☐ 2D ☐ 3D

microscope: ☐ Keyence ☐ Olympus

experiment:

thresholds:  ☐ Auto

fields of view:

---

number of frames:

distance:

Run Ezra's Algorithm?: ☐

Select frames?: ☐

Figure 8: expanded 3D image data settings

image type: ☒ 2D ☐ 3D

microscope: ☒ Keyence ☐ Olympus

experiment:

thresholds:  ☒ Auto

fields of view:

---

number of frames:

distance:

Run Ezra's Algorithm?: ☐ ☒

Select frames?: ☒

Figure 9: example layout of auto-filled settings

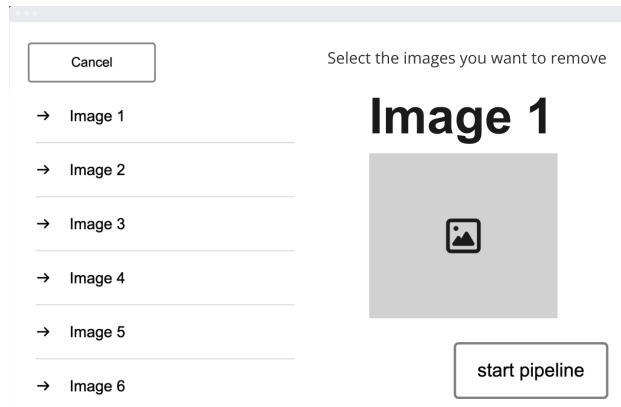


Figure 10: image preview menu

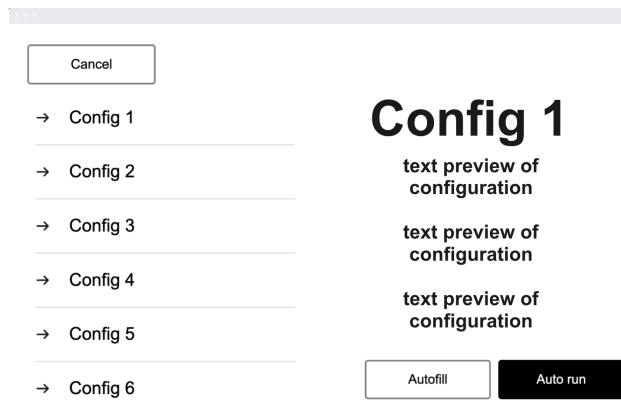


Figure 11: configuration preview menu

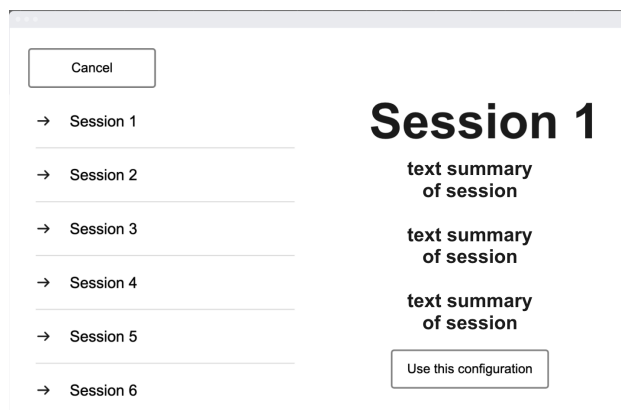


Figure 12: session history menu

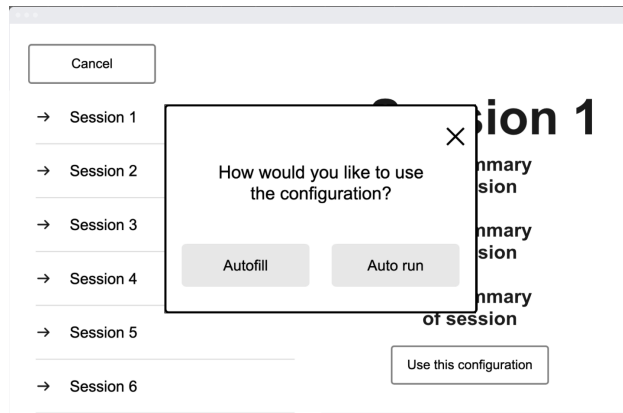


Figure 13: Session history preview with popup for starting a session with contained configuration

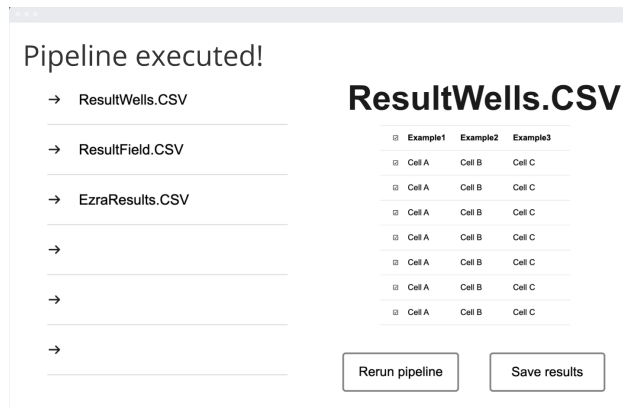


Figure 14: result preview with example result files

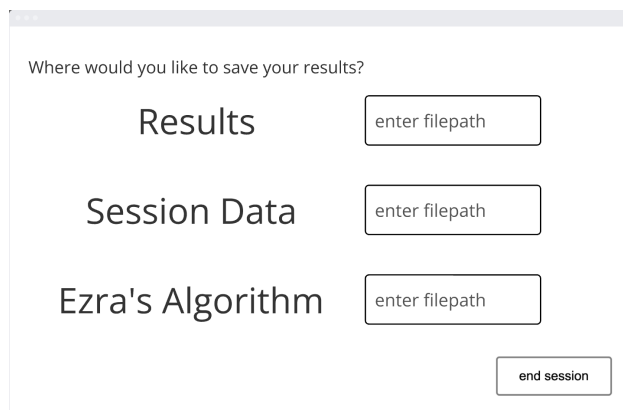


Figure 15: result saving menu

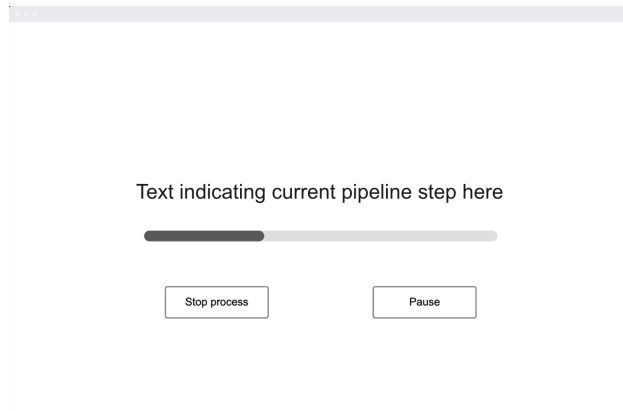


Figure 16: loading screen for running pipeline example

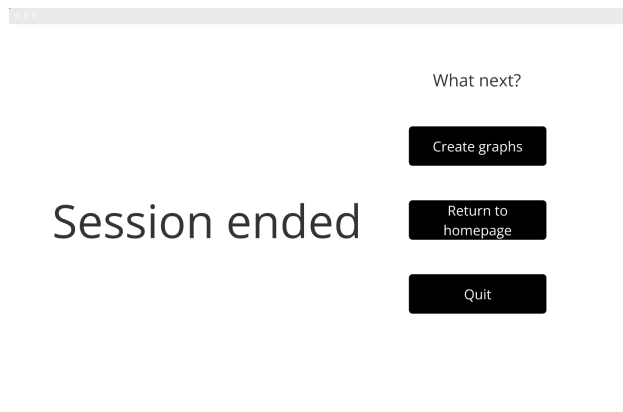


Figure 17: session end screen

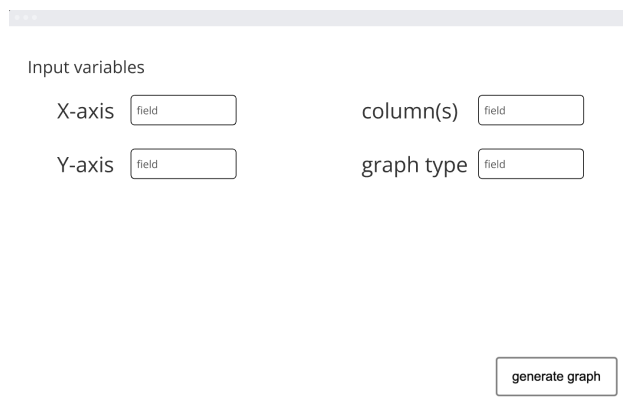


Figure 18: graph creation menu

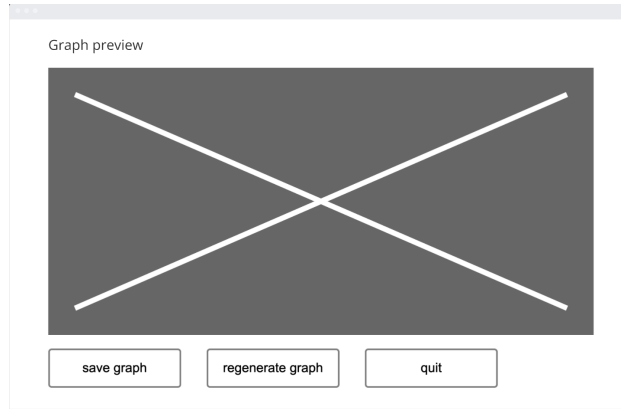


Figure 19: graph preview

## 5 Iterations

### 5.1 iteration 1: Pipeline foundations (Delivery: Feb 10th)

planned:

| ID                  | Title                | Points    | Description                                          |
|---------------------|----------------------|-----------|------------------------------------------------------|
| U1                  | Data upload          | 1         | Upload data files or folders for processing.         |
| U2                  | Setting inputs       | 2         | Input customizable data-processing settings.         |
| U7                  | 2D or 3D pipelines   | 1         | Choose between 2D and 3D pipelines.                  |
| U11                 | Microscope selection | 1         | Select which microscope produced the data.           |
| U21                 | Session start        | 1         | Start a data-processing session.                     |
| S0                  | File organization    | 2         | Create a uniform file structure for raw image files. |
| S1                  | File renaming        | 1         | Automatically rename files for uniform scaffolding.  |
| S5                  | Start date and time  | 1         | Record the start time of a session.                  |
| S14                 | Input validation     | 1         | Validate all user-provided inputs.                   |
| <b>Total Points</b> |                      | <b>11</b> |                                                      |

finished:

| ID                  | Title                | Points   | Description                                         |
|---------------------|----------------------|----------|-----------------------------------------------------|
| U1                  | Data upload          | 1        | Upload data files or folders for processing.        |
| U2                  | Setting inputs       | 2        | Input customizable data-processing settings.        |
| U7                  | 2D or 3D pipelines   | 1        | Choose between 2D and 3D pipelines.                 |
| U11                 | Microscope selection | 1        | Select which microscope produced the data.          |
| U21                 | Session start        | 1        | Start a data-processing session.                    |
| S1                  | File renaming        | 1        | Automatically rename files for uniform scaffolding. |
| S5                  | Start date and time  | 1        | Record the start time of a session.                 |
| <b>Total Points</b> |                      | <b>8</b> |                                                     |

pushed stories:



| ID                  | Title             | Points   | Description                                          |
|---------------------|-------------------|----------|------------------------------------------------------|
| S0                  | File organization | 2        | Create a uniform file structure for raw image files. |
| S14                 | Input validation  | 1        | Validate all user-provided inputs.                   |
| <b>Total Points</b> |                   | <b>3</b> |                                                      |

## 5.2 iteration 2: image processing (Delivery: Feb 24th)

Planned:

| ID                  | Title                | Points    | Description                                         |
|---------------------|----------------------|-----------|-----------------------------------------------------|
| U3                  | Channel selection    | 1         | Select one or more channels for 2D experiments.     |
| U4                  | Image preview        | 2         | Preview images after organization.                  |
| U5                  | Select images        | 1         | Select organized images for a run.                  |
| U6                  | Delete images        | 1         | Delete organized images without reorganizing files. |
| U8                  | Intensity threshold  | 1         | Set or auto-select an intensity threshold.          |
| U9                  | Field of view amount | 1         | Specify number of fields of view per well.          |
| U33                 | SSH access           | 1         | Access files directly via SSH.                      |
| S2                  | FOV aggregation      | 1         | Aggregate all FOVs into a single summary file.      |
| S8                  | Debris calculation   | 1         | Calculate debris when the 2D pipeline is selected.  |
| <b>Total Points</b> |                      | <b>10</b> |                                                     |

Finished:

| ID                  | Title                | Points   | Description                                          |
|---------------------|----------------------|----------|------------------------------------------------------|
| U3                  | Channel selection    | 1        | Select one or more channels for 2D experiments.      |
| U4                  | Image preview        | 2        | Preview images after organization.                   |
| U8                  | Intensity threshold  | 1        | Set or auto-select an intensity threshold.           |
| U9                  | Field of view amount | 1        | Specify number of fields of view per well.           |
| S0                  | File organization    | 2        | Create a uniform file structure for raw image files. |
| S14                 | Input validation     | 1        | Validate all user-provided inputs.                   |
| <b>Total Points</b> |                      | <b>8</b> |                                                      |

Pushed:

| ID                  | Title              | Points   | Description                                         |
|---------------------|--------------------|----------|-----------------------------------------------------|
| U5                  | Select images      | 1        | Select organized images for a run.                  |
| U6                  | Delete images      | 1        | Delete organized images without reorganizing files. |
| S2                  | FOV aggregation    | 1        | Aggregate all FOVs into a single summary file.      |
| S8                  | Debris calculation | 1        | Calculate debris when the 2D pipeline is selected.  |
| <b>Total Points</b> |                    | <b>4</b> |                                                     |

## 5.3 iteration 3: configuration management (Delivery: March 17th)

Planned:

| ID                  | Title                      | Points    | Description                                             |
|---------------------|----------------------------|-----------|---------------------------------------------------------|
| U10                 | Channel selection (stains) | 1         | Select channels used for staining and debris reduction. |
| U12                 | Create configuration       | 2         | Create reusable processing configurations.              |
| U13                 | Read configuration         | 2         | View saved configurations.                              |
| U14                 | Update configuration       | 2         | Modify saved configurations.                            |
| U15                 | Delete configuration       | 2         | Remove unused configurations.                           |
| U16                 | Z-stack frames             | 1         | Specify frames per z-stack.                             |
| U17                 | Frame distance             | 1         | Specify distance between frames.                        |
| S3                  | Wrapping analysis          | 1         | Generate wrapping analysis for 3D data.                 |
| S4                  | Ezra's algorithm           | 2         | Run Ezra's algorithm when selected.                     |
| <b>Total Points</b> |                            | <b>14</b> |                                                         |

**Finished:**

| ID                  | Title                      | Points    | Description                                             |
|---------------------|----------------------------|-----------|---------------------------------------------------------|
| U10                 | Channel selection (stains) | 1         | Select channels used for staining and debris reduction. |
| U12                 | Create configuration       | 2         | Create reusable processing configurations.              |
| U13                 | Read configuration         | 2         | View saved configurations.                              |
| U16                 | Z-stack frames             | 1         | Specify frames per z-stack.                             |
| U17                 | Frame distance             | 1         | Specify distance between frames.                        |
| S4                  | Ezra's algorithm           | 2         | Run Ezra's algorithm when selected.                     |
| U34                 | Mask creation              | 2         | Create masks for 2D and 3D images.                      |
| S9                  | Overlapping calculation    | 2         | Calculate overlapping myelin.                           |
| S10                 | Myelin growth calculation  | 2         | Calculate myelin growth metrics.                        |
| <b>Total Points</b> |                            | <b>15</b> |                                                         |

**Pushed:**

| ID                  | Title                | Points   | Description                             |
|---------------------|----------------------|----------|-----------------------------------------|
| U14                 | Update configuration | 2        | Modify saved configurations.            |
| U15                 | Delete configuration | 2        | Remove unused configurations.           |
| S3                  | Wrapping analysis    | 1        | Generate wrapping analysis for 3D data. |
| S4                  | Ezra's algorithm     | 2        | Run Ezra's algorithm when selected.     |
| <b>Total Points</b> |                      | <b>7</b> |                                         |

## 5.4 iteration 4: 3D image processing (Delivery: March 31st)

**Planned:**

| ID  | Title                 | Points | Description                                           |
|-----|-----------------------|--------|-------------------------------------------------------|
| U19 | Preview configuration | 1      | Preview configuration metadata before selection.      |
| U22 | Session end           | 2      | End or stop a running session.                        |
| U23 | Session history       | 2      | View metadata of previous sessions.                   |
| U24 | Choose configuration  | 1      | Select a saved configuration before a run.            |
| U25 | Auto-fill             | 1      | Automatically run the pipeline using a configuration. |
| U26 | Rerun                 | 1      | Reuse settings from a previous run.                   |
| U27 | Pause a run           | 1      | Pause or stop a running pipeline.                     |
| U28 | Image recovery        | 1      | Recover images excluded from processing.              |

| ID                  | Title                     | Points    | Description                                |
|---------------------|---------------------------|-----------|--------------------------------------------|
| U29                 | Progress monitor          | 1         | View current pipeline execution stage.     |
| S6                  | Script runtime            | 1         | Record runtime of executed scripts.        |
| S7                  | End date and time         | 1         | Record session completion time.            |
| S13                 | Session history           | 1         | Record executed and failed pipeline steps. |
| S15                 | Graceful failure handling | 2         | Preserve partial results if a stage fails. |
| <b>Total Points</b> |                           | <b>16</b> |                                            |

**Finished:**

| ID                  | Title                 | Points    | Description                                           |
|---------------------|-----------------------|-----------|-------------------------------------------------------|
| U19                 | Preview configuration | 1         | Preview configuration metadata before selection.      |
| U22                 | Session end           | 2         | End or stop a running session.                        |
| U24                 | Choose configuration  | 1         | Select a saved configuration before a run.            |
| U25                 | Auto-fill             | 1         | Automatically run the pipeline using a configuration. |
| U27                 | Pause a run           | 1         | Pause or stop a running pipeline.                     |
| U28                 | Image recovery        | 1         | Recover images excluded from processing.              |
| U29                 | Progress monitor      | 1         | View current pipeline execution stage.                |
| S3                  | Wrapping analysis     | 1         | Generate wrapping analysis for 3D data.               |
| S4                  | Ezra's algorithm      | 2         | Run Ezra's algorithm when selected.                   |
| S6                  | Script runtime        | 1         | Record runtime of executed scripts.                   |
| S7                  | End date and time     | 1         | Record session completion time.                       |
| <b>Total Points</b> |                       | <b>16</b> |                                                       |

**Pushed:**

| ID                  | Title                     | Points   | Description                                |
|---------------------|---------------------------|----------|--------------------------------------------|
| U23                 | Session history           | 2        | View metadata of previous sessions.        |
| U26                 | Rerun                     | 1        | Reuse settings from a previous run.        |
| S13                 | Session history           | 1        | Record executed and failed pipeline steps. |
| S15                 | Graceful failure handling | 2        | Preserve partial results if a stage fails. |
| <b>Total Points</b> |                           | <b>6</b> |                                            |

## 5.5 iteration 5: final outputs (Delivery: April 14th)

**Planned:**

| ID  | Title                      | Points | Description                                          |
|-----|----------------------------|--------|------------------------------------------------------|
| U18 | Text file generation       | 2      | Generate editable script text files.                 |
| U20 | Preview cherry-pick images | 1      | Preview images before 3D selection.                  |
| U30 | Export failed files        | 3      | Export scripts with previous settings after failure. |
| U31 | Preview results            | 2      | Preview results before ending a session.             |
| U32 | Preview graphs             | 5      | Generate graphs to visualize results.                |
| U34 | Mask creation              | 2      | Create masks for 2D and 3D images.                   |
| S9  | Overlapping calculation    | 2      | Calculate overlapping myelin.                        |
| S10 | Myelin growth calculation  | 2      | Calculate myelin growth metrics.                     |
| S11 | Calculation results        | 3      | Consolidate outputs into CSV files.                  |
| S12 | Ezra's results             | 1      | Merge Ezra's results into final output.              |

| ID                  | Title | Points | Description |
|---------------------|-------|--------|-------------|
| <b>Total Points</b> |       |        | <b>23</b>   |

**Finished:**

| ID                  | Title                      | Points | Description                                |
|---------------------|----------------------------|--------|--------------------------------------------|
| U20                 | Preview cherry-pick images | 1      | Preview images before 3D selection.        |
| U31                 | Preview results            | 2      | Preview results before ending a session.   |
| S11                 | Calculation results        | 3      | Consolidate outputs into CSV files.        |
| S12                 | Ezra's results             | 1      | Merge Ezra's results into final output.    |
| U23                 | Session history            | 2      | View metadata of previous sessions.        |
| U26                 | Rerun                      | 1      | Reuse settings from a previous run.        |
| S13                 | Session history            | 1      | Record executed and failed pipeline steps. |
| <b>Total Points</b> |                            |        | <b>11</b>                                  |

**Cut stories across all iterations:**

| ID                  | Title                               | Points | Description                                                                                                                           |
|---------------------|-------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------|
| U4                  | Image preview after organization    | 2      | As a user, I want to preview selected images after organization so that I can determine which images should be processed.             |
| U5                  | Selecting images after organization | 1      | As a user, I want to select organized images for a run so that I can clean up experimental image datasets without reorganizing files. |
| U6                  | Deleting images after organization  | 1      | As a user, I want to delete organized images for a run so that I can clean up experimental image datasets without reorganizing files. |
| U18                 | Text file generation                | 2      | Generate editable script text files.                                                                                                  |
| U20                 | Preview cherry-pick images          | 1      | Preview images before 3D selection.                                                                                                   |
| U30                 | Export failed files                 | 3      | Export scripts with previous settings after failure.                                                                                  |
| U32                 | Preview graphs                      | 5      | Generate graphs to visualize results.                                                                                                 |
| U33                 | SSH access for files                | 1      | As a user, I want to access my files via SSH so that I can retrieve them directly.                                                    |
| <b>Total Points</b> |                                     |        | <b>23</b>                                                                                                                             |

## 5.6 Iteration/Sprint 1

### 5.6.1 Planning

For this iteration we planned for the following stories:

| ID           | Title                | Points | Description                                          |
|--------------|----------------------|--------|------------------------------------------------------|
| U1           | Data upload          | 1      | Upload data files or folders for processing.         |
| U2           | Setting inputs       | 2      | Input customizable data-processing settings.         |
| U7           | 2D or 3D pipelines   | 1      | Choose between 2D and 3D pipelines.                  |
| U11          | Microscope selection | 1      | Select which microscope produced the data.           |
| U21          | Session start        | 1      | Start a data-processing session.                     |
| S0           | File organization    | 2      | Create a uniform file structure for raw image files. |
| S1           | File renaming        | 1      | Automatically rename files for uniform scaffolding.  |
| S5           | Start date and time  | 1      | Record the start time of a session.                  |
| S14          | Input validation     | 1      | Validate all user-provided inputs.                   |
| Total Points |                      | 11     |                                                      |

### 5.6.2 Work Done

Total: 11 pts Chloe was assigned: U2, U7, U11, and S14 (5 pts) incomplete story: S14 Chloe's notes: Most of my user stories are done. You can input settings. Some settings from the mockup are purposely missing since they have their own separate user stories and specific interfaces. S14 which was input validation didn't get done due to the iteration itself being cut short as well as required further discussion with the client on input limits and user-directed validation vs system validation.

Jonathan was assigned: U1, U21, S0, S1 and S5 (6 pts) Jonathan's notes: Most of my user stories are complete as the user is able to upload data, raw, and cleaned folders and saved the paths into a json file. Able to call the saved paths using jq in the bash scripts. The run button records the start time and runs step 1 and step 2 of a single channel. And for the session start I was able to figure out how to run the bath files inside tkinter which will be helpful running a session using the pipeline.

### 5.6.3 Testing Coverage

Testing this iteration was a little beyond us for this iteration. For Chloe's assignment, her work was very frontend-based and had little backend function that normal unit testing would work with via pytest. Attempts were made to use pytest on Chloe's tkinter code but proved to be difficult and did not run at all. Jonathan's code was also difficult to test since the bash scripting and renaming cannot be tested by pytest or python unit testing in general. Further deliberation upon our testing methods is being worked on at this time.

Figure 20 shows the test coverage

```
(pyside2) chloemiranda@MacBookPro ArtificialAxonLabPipeline % pytest --cov
===== test session starts =====
platform darwin -- Python 3.10.7, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/chloemiranda/capstone/ArtificialAxonLabPipeline
plugins: cov-7.0.0
collected 0 items

===== tests coverage =====
coverage: platform darwin, python 3.10.7-final-0

Name                Stmts   Miss  Cover
-----
tests/test_SettingFront.py    0     0   100%
TOTAL                        0     0   100%

===== no tests ran in 0.02s =====
```

Figure 20: Iteration 1 test coverage report

Note from Chloe: This is a recovered test file from this iteration. I was looking through my trash and managed to find the test file. I also realized it wasn't running because at that time I didn't have the proper virtual environment set up. I ran perfectly when I redid that test suite.

### 5.6.4 Retroerspective & Reflection

Chloe: This iteration felt a little rough which was purposeful since it was a lot of learning and setting up. For both of us, we were working with some new libraries. This next iteration should be smoother since we're returning to more classic python coding and utilizing Python's imageJ library. For me, the hardest part was learning how to make nice Python code with tkinter. It was very easy to make messy code with little integrity in it, so when it came time to integrating user story U7 with my existing code, I had to do a giant refactor of the whole file just to support one particular user interface. It was for the better because that refactor definitely taught me what I needed to know for this next iteration in working with python. I don't think we are behind just yet since we literally just started, but there is a bit of catching up to do with validating inputs and testing. In terms of how to avoid these learning pitfalls, I guess it would be to already strive for nice code which was hard since I was new and learning how to use this library for our UI but I now know the structure tkinter uses so I don't think I'll be making that mistake next time.

Jonathan: This iteration started off slow trying to explore different options of how to save the folder paths and seamlessly call them in the bash files. First looking into saving the a txt file and then using a json file with jq to call the saved paths. And figuring out how to call the bash scripts inside tkinter. Although I still have to finalize saving the folders and data for step 2 file organization and see if I can improve the flow from one step to another or even automate it, instead of uploading the folder paths again. So it is interesting thinking about what you are doing and how it would impact other stories and the overall pipeline which is important so we do not have to go back and re-facture large parts of the pipeline. And for testing I was able to create another bash script of seeing how I could save the folder paths and call them as a variable that I would use in the pipeline and using jq and the json file I was able to print out all of the saved folder paths that ran from a tkinter button. And to test the pipelines I would need to obtain sample raw data from our client to make sure the folder paths are properly being implemented during a run even though i was able to print out the file paths and the pipeline did not change. It is still helpful to know that step 1 and 2 will run successfully during an experiment.

## 5.7 Iteration/Sprint 2

### 5.7.1 Planning

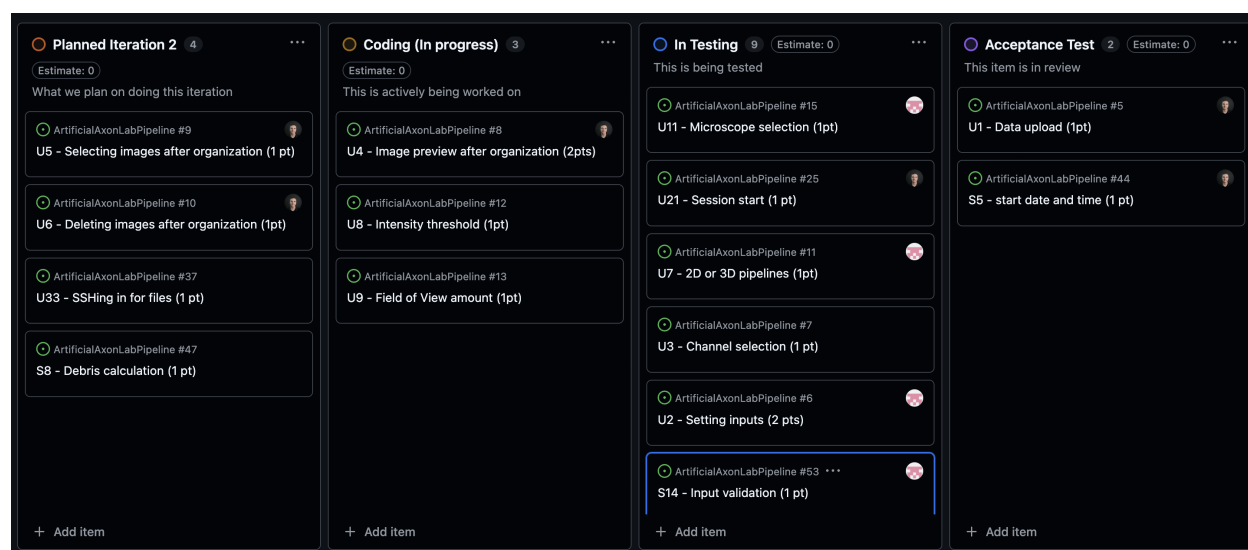


Figure 21: iteration 2 kanban

The stories for this iteration mainly focused on finishing off the file renaming and organization and any inputs needed for that. This totaled up to 13 points, which meant the 10 points from the original iteration and 3 points from the stories we didn't get to in iteration 1.

### 5.7.2 Work Done

Chloe was assigned U3, U8, U9, S2, S8 (6 pts)

Chloe's notes: I was able to complete U3, U9, and U8. The stories that did not get done dealt a lot with PyImageJ and I wanted to prioritize getting the file organization and renaming done first as it is the first step of the pipeline. Along with that, I believe this next iteration needs a little bit of deliberation with the client as U33 might not be necessary anymore, as well as S2 since aggregation inherently happens during the renaming process.

Jonathan was assigned U4, U5, U6 (4 pts)

Jonathan's notes: In this iteration I was able to work on some previous user stories of File renaming and File organization exploring some options of using a script to get file names. And I was able to get part of Image preview done testing it on a folder of .tiff images. And i was not able to get to selecting and deleting the images because most of my time was spend researching and initializing the use of pyimage. Creating a clear document in the dicussions tab in Github on command and how to run pyimage / fiji.

### 5.7.3 Testing Coverage

```
===== tests coverage =====
coverage: platform darwin, python 3.10.7-final-0

Name                Stmts   Miss  Cover
-----
view/SettingFront.py    90      90      0%
view/UploadPageStep1.py 133      46     65%
view/UploadPageStep2.py  45       1     98%
view/__init__.py         0       0    100%
TOTAL                  268     137     49%

===== short test summary info =====
FAILED tests/test_step1.py::test_save_folders_classifies_directories - FileNotFoundError: [Errno 2] No such file or directory: 'folder_paths.json'
FAILED tests/test_step1.py::test_save_folders_sorts_tracks - FileNotFoundError: [Errno 2] No such file or directory: 'folder_paths.json'
FAILED tests/test_step1.py::test_start_time_adds_timestamp - FileNotFoundError: [Errno 2] No such file or directory: 'folder_paths.json'
FAILED tests/test_step1.py::test_run_step1_windows - AssertionError: assert False
FAILED tests/test_step1.py::test_run_step1_unix - AssertionError: assert False
FAILED tests/test_step1.py::test_channels_convert_to_json_format - FileNotFoundError: [Errno 2] No such file or directory: 'folder_paths.json'
===== 6 failed, 10 passed in 0.58s =====
```

Figure 22: Iteration 2 testing coverage report

Because Tkinter elements which do not need testing are in the same file, coverage may not technically be as high, but regardless is still acceptable.

### 5.7.4 Retroespective & Reflection

Chloe: This iteration was a little rough. We're running a little behind, but this is mainly because of PyImageJ and some time management issues. After conversation with each other and the client, the backlog isn't concerning as the issues are clear and fixable. We also realized that it would be better to have more than one meeting a week, and we will do so going forward.

Jonathan: This iteration was interesting working with Fiji which is just pyimage with additional plugins and trying to get everything setup looking into different options of how to run it and and all of the different dependencies you needed to install to run it like miniconda and making sure you add the path to the terminal and initializing Fiji. This took a very long time to research and sort out and then later looking into how to run it in a virtual environment instead of your local terminal environment and installing all of the necessary packages. Although this took most of my time, I am glad it was well documented so myself and others can read it and understand how to setup Fiji and the virtual environment, and i was able to get pyimage to open .tiff images in a folder by using python and initializing an external Fiji so that it can be always up to date which was surprising because before i tried it, Fiji never loaded.

## 5.8 Iteration/Sprint 3

### 5.8.1 Planning

Chloe: U10, U17, U34, S9, S10, S3 (8 pts)

Jonathan: U12, U13, U14, U15 (8 pts) total: 16 points

Note: some stories were pushed to later iterations: U16, S4 other stories were pushed up to now: S9, S10, U34

### 5.8.2 Work Done

Chloe: I was able to get anything involving masking done and any inputs with masking. This means U34, S10. So, just a little over half of my points were completed. S9 is partially done, the masks for it are made, but the actual calculation is not. I will take note to mention that I'm greatly debating increasing the amount of points that U34 (creating masks). The effort it took to figure out the libraries necessary for the code I wrote took much more than two weeks. I had started it in a separate folder on my own time to try and get ahead, which is why that work isn't recorded on the repository, but it took more than two weeks to create what I had pushed for this iteration. Testing it was even more of a pain. If I were to estimate it now, I'd make the U34 user story 5 points, that's how painful it was. You can check my commit messages and sporadic comments if you want to know how painful it was to make that.

Jonathan: In this iteration I was able to create a tinkter list box that is able to pull from a configuration folder and list the current saved configurations. And when clicking on a configuration and pressing the delete button it gives you a warning if you want to delete it, so you do not accidently delete it with just a click of a button. And I was able to move all of the saved information from the run like the well, track, and dir list and json file of settings into a data folder instead of it just dumping into the current view folder. Where you are able to save the current configuration in the data folder and copies it into the configuration folder with the name that you have set. But i am in the process of running through some ideas of how to edit a configuration and being able to run it with other data as you would want to organize your data but not want to reorganize it with the already organized data. And when working with new data you do not want the saved configuration to be pointing to old data.

### 5.8.3 Testing Coverage

| Name                                                                           | Stmts | Miss | Cover |
|--------------------------------------------------------------------------------|-------|------|-------|
| /Users/chloemiranda/capstone/ArtificialAxonLabPipeline/analysis/masking.py     | 163   | 18   | 89%   |
| /Users/chloemiranda/capstone/ArtificialAxonLabPipeline/view/UploadPageStep1.py | 139   | 42   | 70%   |
| /Users/chloemiranda/capstone/ArtificialAxonLabPipeline/view/__init__.py        | 0     | 0    | 100%  |
| conftest.py                                                                    | 16    | 0    | 100%  |
| test_masking.py                                                                | 144   | 0    | 100%  |
| test_step1.py                                                                  | 94    | 0    | 100%  |
| TOTAL                                                                          | 556   | 60   | 89%   |

Figure 23: Iteration 3 testing coverage report

### 5.8.4 Retroespective & Reflection

Chloe: The biggest challenge of this was that stupid PyImageJ library. It is very picky on what inputs it needs, most of the debugging of this was figuring out what it wants just so it can run and tell me I'm missing something else. The documentation for it is also painfully unhelpful. I had to go down a long rabbit hole to realize I needed a whole other library, scyimage, to run pyimagej. In terms of avoiding things, I can't really avoid this awful, awful library. I did however, learn a lot about image processing and why these researchers used their old pipeline in such a specific way. For example, order really matters and I did my best to maintain that in this code. The true myelin mask is actually created from subtracting the debris



mask from the raw myelin mask, which I thought was interesting, and having tools that are cool in practice. Implementing said tools in code is much less cool

Jonathan: My biggest conflict was thinking about the overall design of our architecture and how everything would work together because when re-running a configuration you only want to run step 2 and forward. What should be saved in the configuration settings when dealing with the folder paths as using new data with an old configuration you do not want it to be pointing to old data. Despite this i was able to create a system to load and save configurations of putting it in the configurations folder or moving it up to the configuration that is going to be used in the data folder.

## 5.9 Iteration/Sprint 4

### 5.9.1 Planning

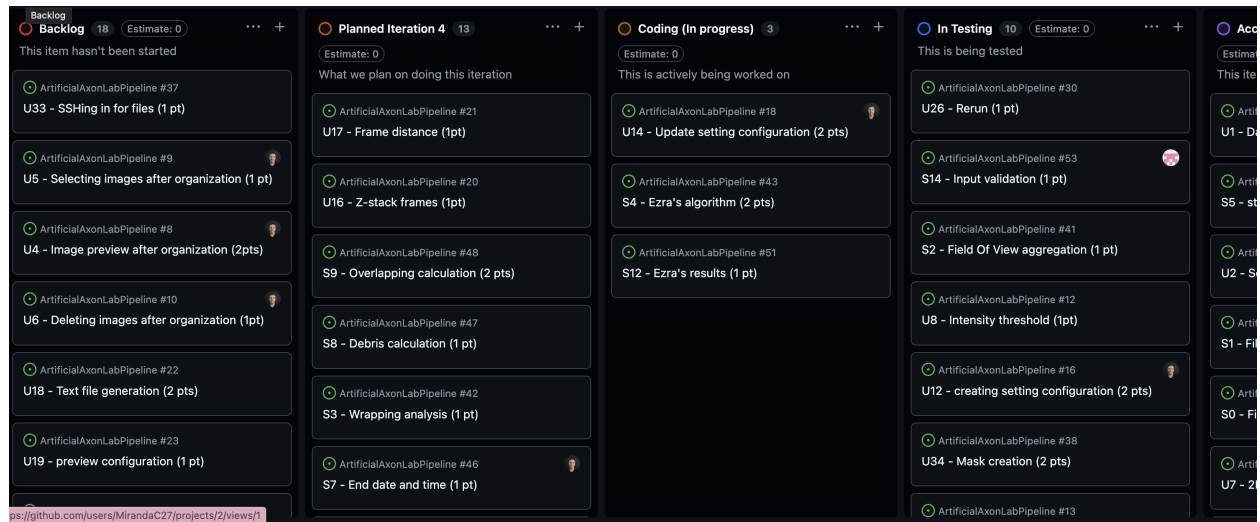


Figure 24: iteration 4 kanban

This iteration was really to finish off the pipeline and either get started on the extra features they need or to tie up any loose ends on features that were already started.

### 5.9.2 Work Done

Pushed stories: S15 graceful failure handling got pushed. We are going to do more integration testing for this upcoming iteration so graceful failure handling will be easier to integrate.

Chloe was assigned: S3, S4, S12, S8, S9, U16, U17 (7 points)

incomplete story: none. they all got done this time :)

Chloe's notes: The story that got pushed to now took up most of this iteration and I'm going to take the time to explain why. In the previous iteration, I mentioned that the PyImageJ library is very picky. The reason that story got pushed was because I needed to focus on figuring out how to do some complicated things with its basic functions. The stories that got pushed to this one specifically work with the 3D image processing functions of PyImageJ and were even worse. I believe the only reason that these didn't take as long as figuring out masking from the previous iteration was simply because the code didn't need as many functions compared to masking and I had a little bit of knowledge to jump off from. Ezra's algorithm also works... there's not much to say about that. In all honesty it's probably the most normal python code in this whole project.

Jonathan was assigned: U19, U22, U23, U25, U29, S6, S7 (7 pts)

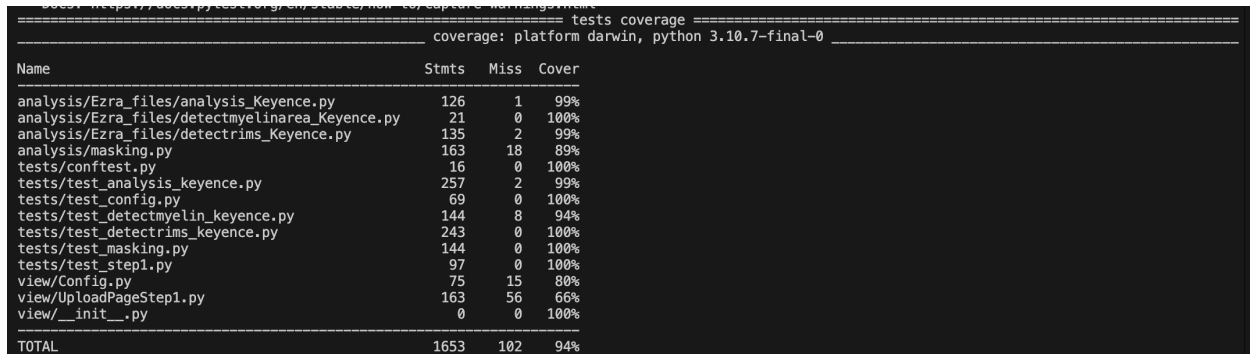
incomplete story: U23, S13, U26

Jonathan's notes:

I was able to complete most of these stories but when thinking about the pause a run I started to question

the usefulness of this as there would be nothing you would need to edit or do in a middle of a run. And For Session history I need to create a storage system like for the configurations that stores the run and some basic info like image type, and microscope used and order them so you have a general idea what was being used in the last run.

### 5.9.3 Testing Coverage



| Name                                            | Stmts       | Miss       | Cover      |
|-------------------------------------------------|-------------|------------|------------|
| analysis/Ezra_files/analysis_Keyence.py         | 126         | 1          | 99%        |
| analysis/Ezra_files/detectmyelinarea_Keyence.py | 21          | 0          | 100%       |
| analysis/Ezra_files/detectrimms_Keyence.py      | 135         | 2          | 99%        |
| analysis/masking.py                             | 163         | 18         | 89%        |
| tests/conftest.py                               | 16          | 0          | 100%       |
| tests/test_analysis_keyence.py                  | 257         | 2          | 99%        |
| tests/test_config.py                            | 69          | 0          | 100%       |
| tests/test_detectmyelin_keyence.py              | 144         | 8          | 94%        |
| tests/test_detectrimms_keyence.py               | 243         | 0          | 100%       |
| tests/test_masking.py                           | 144         | 0          | 100%       |
| tests/test_step1.py                             | 97          | 0          | 100%       |
| view/Config.py                                  | 75          | 15         | 80%        |
| view/UploadPageStep1.py                         | 163         | 56         | 66%        |
| view/__init__.py                                | 0           | 0          | 100%       |
| <b>TOTAL</b>                                    | <b>1653</b> | <b>102</b> | <b>94%</b> |

Figure 25: Iteration 4 test report

### 5.9.4 Retroespective & Reflection

Chloe: I thought the other iteration was painful, this was even worse. I think the person who made the 3D viewer in ImageJ is a sadist. I thought that the macro masking in PyImageJ was picky. I was wrong. This was so much worse. On a side note, I didn't realize I already completed two user stories that were supposed to be for this iteration so that was a nice win. Anyways, I don't think there was any way to avoid using the 3D image viewer. If I could, I would. In terms of not going according to plan, I certainly wasn't planning for dealing with 3D image objects to be this painful, and I now know there is no way to avoid it. I'm just ready to jump into system testing for this next iteration. Like the other story, I would've made this way more points, at least 5, maybe 3 since it was technically less code. Those two extra points would be more representative to the emotional costs of these stories.

Jonathan: During this iteration it was very interesting trying to create the different pages because I was initially setting them up to run independently to see what they looked like for the Configurations page and then in order to implement the auto fill feature I had to make it switch to the upload page which was not loaded. So in order to load the page I had to make the page a tkinter frame and add it to the main page and adjust all of the navigation buttons to get it to the page. And for coding the auto fill feature I wanted it to be in its own utility file because it was going to include functions not dealing with the ui of the page and focused on this important feature. So in order to know what to enter into the upload page you first have to read the selected json file in the list and extract the data in the input fields that are needed to fill in the page where a function in the upload page takes that data in and sets each field. So the utility function is only dealing with the data while the function in the page is taking that data and setting in the tkinter inputs. And whenever i tested the step 1 script it was very concerning because I implemented parallel processing and it completes significantly faster but my laptop gets extremely hot to the point I had to unplug it and wait for it cooled off before I continued testing. So I had to limit the number of jobs it was creating so that it did not overload and eventually kill my laptop at the cost of 2 minutes added to the runtime which I say is a fair tradeoff.

## 5.10 Iteration/Sprint 5

### 5.10.1 Planning

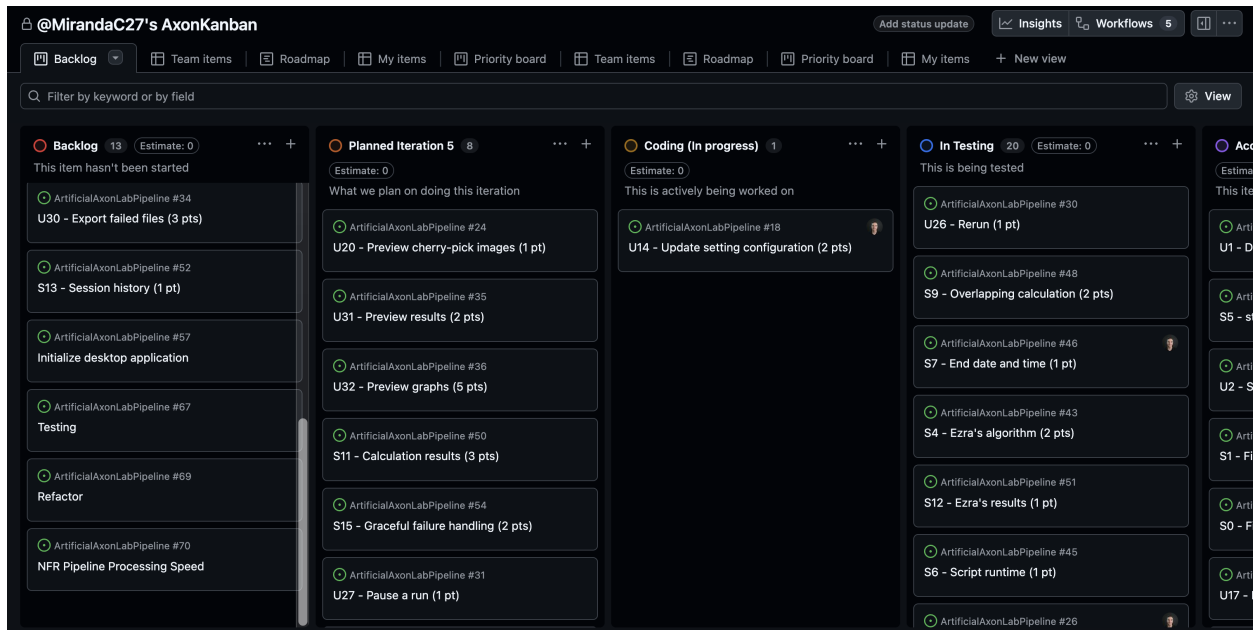


Figure 26: Iteration 5 kanban board

### 5.10.2 Work Done

Chloe was assigned: U31, U32, S11, S12, S15, planned refactoring (2ish points worth of effort) (7 points)  
incomplete story: S15, U32  
Chloe's notes: For S15, In terms of failure, it won't crash, it'll let you know and still allow you to go through the pipeline, but the failure isn't integrated in the UI. U32 I just didn't get to it. It was already tricky refactoring this so that it prints out a smooth csv for U31.

Jonathan was assigned: U20, U23, (4 points)

Jonathan's notes: I was able to implement cherry picking of FOV's and channels with a clean Ui interface, but still ironing out the last bit of logic for sessions history to only save a certain amount of sessions

### 5.10.3 Testing Coverage

| Name                                            | Stmts | Miss | Cover |
|-------------------------------------------------|-------|------|-------|
| analysis/Ezra_files/analysis_Keyence.py         | 126   | 1    | 99%   |
| analysis/Ezra_files/detectmyelinarea_Keyence.py | 21    | 0    | 100%  |
| analysis/Ezra_files/detectrimms_Keyence.py      | 135   | 2    | 99%   |
| analysis/masking.py                             | 185   | 32   | 83%   |
| controller/AutoFillUtil.py                      | 63    | 54   | 14%   |
| controller/SessionDataUtil.py                   | 74    | 64   | 14%   |
| controller/__init__.py                          | 0     | 0    | 100%  |
| tests/__init__.py                               | 0     | 0    | 100%  |
| tests/conftest.py                               | 16    | 0    | 100%  |
| tests/test_analysis_keyence.py                  | 257   | 2    | 99%   |
| tests/test_config.py                            | 69    | 12   | 83%   |
| tests/test_detectmyelin_keyence.py              | 144   | 8    | 94%   |
| tests/test_detectrimms_keyence.py               | 243   | 0    | 100%  |
| tests/test_masking.py                           | 149   | 0    | 100%  |
| tests/test_step1.py                             | 97    | 38   | 61%   |
| view/Config.py                                  | 157   | 137  | 13%   |
| view/UploadPageStep1.py                         | 224   | 118  | 47%   |
| view/__init__.py                                | 0     | 0    | 100%  |
| view/masking_front.py                           | 174   | 145  | 17%   |
| TOTAL                                           | 2134  | 613  | 71%   |

Figure 27: Iteration 5 test report

Note: ignore the 17% on masking\_front.py. It's actually untested since it's just a frontend and doesn't have any special cases like UploadPageStep1.py. If you removed that the coverage is easily around 80%

### 5.10.4 Retropective & Reflection

Chloe: This iteration was alright, it was easier than the last two but I think it'll still take some final polish afterwards. The graceful error handling I didn't get to integrate will probably be finished after this iteration is due but that's okay. Honestly, I'm not surprised that I didn't get to graphs. In all honesty, that user story was tacked on for a bit of point cushioning and even the client admitted that it's okay if the final version doesn't include it. Along with that, the points that went into making those stupid masks and dealing with the 3d object manager definitely makes up for those lost points.

Jonathan: This iteration was very shocking because it felt like time flew by so quickly, although i was able to implement cherry picking of channels and Fovs but this took me a couple of iterations to get as the tkinter UI was kind of hard to implement with all of the spacing and clicking a checkbox that relegated the disabled Fov box on the right side and just added a disable channel box for the channel inputs that would already be on the page instead of creating and displaying the already inputted channels to disable. And for logic I intensity thought of removing the channels and fovs when creating and organizing the data but I thought of when you would want to enable the channel or fov back and you would have to organize the data again which would take another couple of minutes to organize the data again. So now the logic is when creating the data and the masks so that when you are trying to call the ordered data it goes through it and skips the selected Fov or channel. And for the session history I still need to add the intensity threshold inputs and a way to limit the number of sessions as you would just want to see the most recent and relevant runs that you did. And the page is like the config page where it lists then on the left side and the contents on the right side seeing the most important information about the run

## 6 Final Remarks

### 6.1 Overall Progress

There were a few cut user stories for both time and just common sense usability. Point-wise those were easily made up by the effort that was put into the last two steps of the pipeline: mask creation and data consolidation. Preview graphs did not get done simply because of time spent on masking and Ezra's algorithm. Cherry picking frames got cut as well because upon further discussion with the client that particular step actually happens after the whole pipeline is done and it's more practical for them to delete on their own and re-run the pipeline than to have that step actually inside it. Export failed files got cut just because the bash script got complicated enough that if they did export it, it wouldn't be usable, and it would be easier for them to just revert to their old pipeline. SSHing got cut because they just did not need to ssh anywhere. During iteration 2 we discussed more details about it with the client and realized that they don't actually need to ssh anywhere for their files and it's more practical to have the application access their files locally. This totals to 10 points left on the table. We understand this is a lot, BUT again, this was made up for in the effort it took to create masks, and consolidate data. None of the points that were left on the table take away from the usability of the pipeline. If we were to try and finish up the one story we didn't get to, which is graph generation, We estimate that it would take one more two-week iteration, depending on if we added anything special that the client requests.

### 6.2 Project Reflection

**Chloe:** The biggest lesson I've learned is to never underestimate the pain of image processing. After learning this pipeline and making design choices on what to consolidate I think I would definitely revise how some of these iterations played out. It feels silly to say that since I had submitted a revision of these user stories before this project began and I'd still change them now. I would definitely inflate the point values on a lot of the image processing steps I did. Jonathan and I definitely took the steps to go over the pipeline with our client before the project started and even that was still to prepare myself for the difficulty of this project. For example, I thought I could consolidate a lot of the masking for the pipeline but with the way the PyImage works I had to make way more functions and ended doing more work instead of shortcutting. For the future, I think I would've tried to allocate more time on learning the libraries we used like PyimageJ. I did look over the library and looked at examples of its usage but I might've made some better design choices and had a better idea of how much effort coding in it would take. For example I learned that that it PyImageJ is painfully picky on file names and is probably the reason why their original pipeline has a very strict naming convention. I think that caused a lot of issues design-wise, especially when trying reduce coupling. Learning how to use the 3D object manager in PyImageJ was even worse, and getting that section to simply open was extremely difficult. Overall, I think we just picked a conceptually difficult project. It was combination of both research and software engineering after reflecting upon it and admittedly it tested both my computer science and scientific research knowledge. I've worked with biologists and that experience definitely influenced the design choices we've made especially after deliberating upon pipeline order with my partner.

**Jonathan:** Some of the lessons I learned were how to manage user data and configurations, as well as how to work with and read JSON data. Since it is important to store all data in a central location for consistency and session management, I implemented a single JSON file with IDs instead of creating separate files. Previously, I used separate files for user-saved configurations to save the run settings. In the future, I would like to refactor the configurations to follow the same centralized approach, so that everything is more organized. However, storing configurations in separate files has the advantage of making it easy to drag and drop files into a folder for importing and exporting settings between users. Another lesson I learned was the importance of ensuring that scripts or code work across both Windows and macOS systems, especially due to differences in file path separators. To address this, when linking file paths, we first refer to the script directory and then navigate to the target file. This allows us to rely on a more uniform file structure, similar to what is used in Git Bash, as mentioned in the general information post on the discussion tab, rather than handling OS specific differences directly. Although this approach provides a consistent structure, I wonder if encapsulating the application in a Docker container would better address cross operating system issues.

However, doing so would significantly increase complexity and the risk of errors when building and running the container.

## Reflection on Final Presentation and Demo

Our final project got the job done. I think we both did a good job and it was certainly very different than the other projects. The biggest criticism we received was our UI. I didn't realize that we had forgotten to put a second loading screen for running a demonstrated step of the pipeline. Also, we were asked some questions about GPU in relation to image processing that we were simply unable to answer and forgot to consider. In retrospect I think we should've taken more time to create a better "demo version" with demo data. We did our best to create something reasonable for the day of but I think it could've been better. There were also questions that were asked that were good and allowed us to better explain how the pipeline works. One of the faculty asked why we didn't have any image previews in our UI and we got a chance to explain that our users don't actually look at the images to make decisions, only the data that's produced from them. If they directly look at images they do so after getting data and pick arbitrarily based on the well, and again, do not directly look at the images. I think the positive reception came after we had time to explain what it did and how it actually improved their lives during the demo. It seemed long then, but once we explained in more detail how it consolidates what our clients did before, we got more positive reception. For example, we explained that before our pipeline, our client would need to manually go through each well and run a script on every single individual image, which is very inconvenient with almost 3 gigabytes of images. With our pipeline, it will just automatically iterate over every single well, and every single specific image for them without needing to do anything. In the end,

## AI usage

AI was used to revise user stories

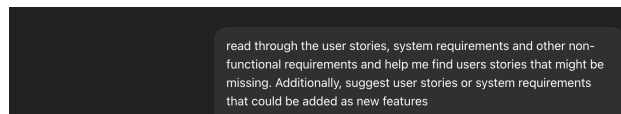


Figure 28: user story message with ChatGPT

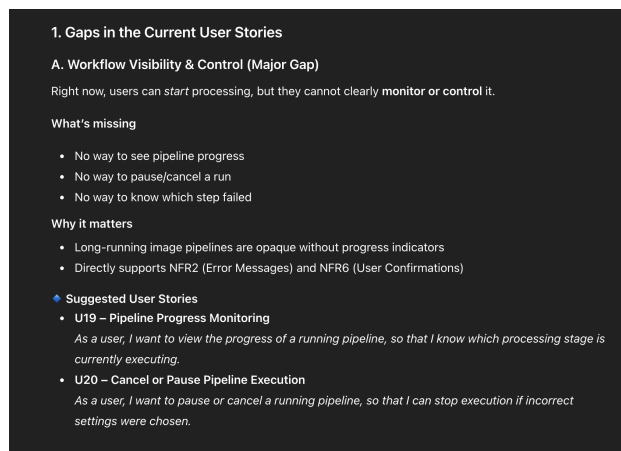


Figure 29: user story response from ChatGPT

### **6.3 reflection**

In terms of helpfulness, it was actually really helpful in finding gaps. It looked at my existing user stories and caught some inherent user stories that were missing. For example, we had start and stop session stories but there was no pause story. It also provided pre-made resulting user stories, which I did not use, and it overestimated points sometimes.